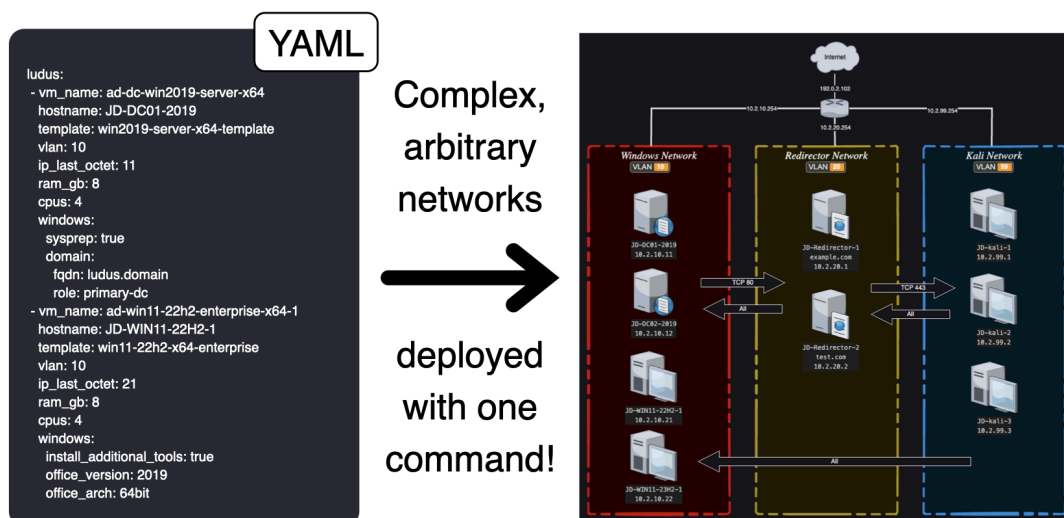


Foray into Ludus.Cloud | Ludus Cheatsheet

 detectx.com.au/ludus-cloud-draft

rockstarL

13 December 2024



I use and contribute to a few different Pen Test Lab environments, including Detection Cloud and Azure version of that, as I have a spare old HP server, thought I try Ludus.cloud.

Building and maintain Cyber Security Lab environments is seriously complicated.

The main problem with building such labs is that there is always something that is not compatible or break. Terraform update breaks a specific driver and your whole lab doesn't work.

I also want everything automated, As I don't need it all the time, but when I do, I want to type a single command and have a very complex environment up and running securely to run malware as an example and not infect myself. Running it on cloud is the best option as if malware escapes you can easily delete the AWS environment quickly. But, I hate paying for any subscriptions of anything. Compute nowadays is so cheap and you can run a massive amount of VM on a 2K AUD NUC. I wonder why people move to the cloud, as Intel CPU, NVMe and RAM is so cheap. Gotta do what the cool kids do..

Setup Ludus on VMware Workstation

- Make sure you PC has enough resources, You need 32 GB or 64 or more and 1 TB nVME Disk and Intel CPU, here is my specs. This was about 1,000 AUD in 20/07/23.
 - CPU: Intel Core i9 – <https://www.umart.com.au/product/intel-core-i9-13900f-24-core-lga-1700-5-60ghz-cpu-processor-bx8071513900f-68199>
 - RAM 64 GB – Corsair 64GB (2x32GB) CMK64GX5M2B5200C40 Vengeance CL40 5200MHz DDR5 DRAM – Black
 - DISK: Samsung 990 Pro 2TB PCIe 4.0 M.2 2280 NVMe SSD (MZ-V9P2T0BW) – <https://www.umart.com.au/product/samsung-990-pro-2tb-pcie-4-0-m-2-2280-nvme-ssd-mz-v9p2t0bw-66090>
 - Motherboard – <https://www.umart.com.au/product/gigabyte-z790m-aorus-elite-ax-d5-lga-1700-matx-motherboard-rev-1-2-ga-z790m-aorus-elite-ax-68408>
 - Power – <https://www.umart.com.au/product/msi-750w-mpg-a750gf-80-gold-power-fully-modular-atx-power-supply-black-mpg-a750gf-57426>
 - Case – <https://www.umart.com.au/product/asus-ap201-prime-micro-atx-case-mesh-white-prime-ap201-wh-65231>
- Setup VMware Workstation

VMware Workstation is now free, Download VMware Workstation using the following Guide – <http://www.detectx.com.au/howto-download-vmware-workstation-pro-and-vmware-fusion-pro-for-free/>
- Disable Hyper-V

In order to create a VM with Virtualized Intel VT-x/EP, do the following on Windows 10 and 11;

 - Run SystemInformation / msinfo32 to identify if Hyper-V is enabled
 - Turn windows Feature On or Off and remove all Hyper-V
 - bcdedit /set hypervisorlaunchtype off
 - WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V-All
 - Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity" -Name "Enabled" -Value 0
 - Reboot
- Create a VM
 - HD 400 GB
 - CPU
 - RAM: use Max Ram
 - Intel VT-x/EP
 - Networking should be Share with Host
 - Setup Static IP for VM code C:\ProgramData\VMware\vmnetdhcp.conf

- Download the Debian 12 ISO – <https://cdimage.debian.org/debian-cd/current/amd64/iso-cd/>
 - If you are installing Debian 12, the [netinst ISO](#) (scroll down to debian-12.x.x-amd64-netinst.iso) is recommended. At the screen below during install, uncheck Debain desktop environment and check SSH server – <https://docs.ludus.cloud/docs/quick-start/install-ludus/>
 - Install Debian with System Utils and SSH only
- Install LUDUS
 - Connect to Proxmox – <https://192.168.193.140:8006>
 - Build Templates
- Build Range
- Access Range

Access Ludus Lab

WireGuard

The easiest way to interact with Ludus VMs is directly - via SSH, RDP, or KasmVNC. This is possible thanks to a WireGuard VPN hosted on the Ludus server. To get the WireGuard configuration file for your user, run the user wireguard command. Admins can get WireGuard configurations for other users with the user wireguard --user <UserID> command.

```
# ludus user wireguard | tee ludus.conf
# ludus user wireguard --user ghost | tee ludus.confuiug

[Interface]
PrivateKey = KBxrT+PFLCII+uJo9a6XLm/b23vbqL5KmNQ5Ac6uwGI=
Address = 198.51.100.2/32

[Peer]
PublicKey = 5nLD06gtqVXI89xQNkd2c2L0US7RnPinbAlfiyWHHBM=
Endpoint = 203.0.113.54:51820
AllowedIPs = 10.2.0.0/16, 198.51.100.1/32
PersistentKeepalive = 25
```

On your local machine, import this WireGuard configuration (ludus.conf) into the WireGuard GUI client or on the command line with wg-quick and connect. wg setconf is not supported by this configuration. Now you can directly interact with range VMs as if you were on the same network.

To get a zip file of RDP configuration files for the Windows machines, use the

range rdp command.

```
#ludus range rdp
```

```
[INFO] File downloaded and saved as rdp.zip
```

These configuration files can be imported into any RDP client and used to connect to the Windows VMs using the credentials below.

All Linux machines have SSH running on port 22.

All Windows machines have RDP enabled on port 3389.

The Kali template is running KasmVNC on

<https://<kali IP>:8444/>. The password for KasmVNC is password as it requires a 6 character password.

```
## Access wireguard
```

```
ludus user wireguard | tee ludus.conf
ludus user wireguard --user ghost | tee ludus.conf.iug
#ludus range rdp
ssh-keygen -R 192.168.1.28
scp root@192.168.1.28:/root/ludus.conf .
ludus range status
```

One user with multiple Labs

```
# One user with multiple ranges
```

One user of Ludus wishes to have separate ranges but control them all with the same WireGuard config and Ludus API key (i.e. a Ludus host dedicated to a single user but with many ranges).

```
# https://docs.ludus.cloud/docs/sharing
```

```
ludus --url https://127.0.0.1:8081 user add -n 'CBC Demo1 Range' -i cbcdemo1
ludus --url https://127.0.0.1:8081 user add -n 'CBC Demo2 Range' -i cbcdemo2
```

```
ludus user list
```

GitHub serves "raw" pages with Cache-Control: max-age=300. That's specified in seconds, meaning the pages are intended to be cached for 5 minutes.

```
rm basic.yaml
curl -S -H 'Cache-Control: no-cache' -o basic.yaml
"https://raw.githubusercontent.com/RockAfeller2013/ludus/refs/heads/main/basic.yaml"
"
```

```

ludus --user cbcdemo1 range config set -f basic.yaml
ludus --user cbcdemo2 range config set -f basic.yaml

ludus --user cbcdemo1 range deploy
ludus --user cbcdemo1 range list
ludus --user cbcdemo1 range status
ludus --user cbcdemo2 range logs -f --verbose

ludus range deploy --limit <vm name>
ludus --user cbcdemo2 range deploy --limit cbcdemo2-WIN10

# Don't deploy all ranges at the same time. It takes basic about 30 mins to build
out

ludus --user cbcdemo1 power on --name all
ludus --user cbcdemo2 power on --name all

ludus --user cbcdemo1 power off --name all

ludus --user cbcdemo1 range rm
ludus --user cbcdemo2 range rm

ludus range access grant --target cbcdemo1 --source SA
ludus range access grant --target cbcdemo2 --source SA

ludus --user <network you want access to> range deploy -t network
ludus --user cbcdemo2 range deploy -t network

ludus range access list

ludus user wireguard --user SA | tee SA.conf

# Import into Wireguard

scp root@192.168.1.28:/root/cbcdemo1.conf

ludus user creds get
ludus version --url https://127.0.0.1:8080

```

Defaults Machine Credentials

PROXMOX ID	VM NAME	POWER	IP	
+-----+	-----	+-----+	-----	+
108	SA-router-debian11-x64	On	10.2.10.254	
109	SA--dc-win2022-server-x64	On	10.2.10.11	
110	SA--win11-22h2-enterprise-x64-1	On	10.2.10.21	
111	SA-kali	On	10.2.99.1	
+-----+	-----	+-----+	-----	+

AdGuard
http://10.2.10.254:3000 admin:password.
http://10.{{ range ID }}.{{ any VLAN specified in user's config }}.254:3000. The
credentials are admin:password.

Kali
https://10.2.99.1:8444/ (kali/password) (kali/kali)
kali:kali (OS)
kali:password (KasmVNC - https port 8444)
Windows
localuser:password (local Administrator)
LUDUS\domainuser:password
LUDUS\domainadmin:password (Domain Admin)
Debian based boxes
debian:debian
Others
localuser:password

Testing Mode

<https://docs.ludus.cloud/docs/quick-start/testing-mode/#denying-previously-allowed-domains-and-ips-during-testing>

ludus testing start

ProdSYD Firewall Configuration

<https://techdocs.broadcom.com/us/en/carbon-black/cloud/carbon-black-cloud-sensors/index/cbc-sensor-installation-guide-tile/GUID-8DD05446-4094-4019-AA0C-D2ED1CB15FC0-en/firewallconfig/prodsyd-firewall-configuration.html>

ludus testing allow -d example.com
ludus testing deny -d example.com
testing status
ludus testing stop

ludus testing allow -d defense-prodsyd.conferdeploy.net,dev-prodsyd.conferdeploy.net,cdc-file-storage-production-ap-southeast-2.s3.amazonaws.com,storage.googleapis.com,psc-jobs-prod-output-ap-southeast-2.s3.ap-northeast-1.amazonaws.com,updates2.cdc.carbonblack.io,ocsp.godaddy.com,ocsp.digicert.com,crl.godaddy.com,crl3.digicert.com,crl4.digicert.com,content.carbonblack.io

ludus testing allow -i 34.117.142.85

ludus testing allow -d defense-prodsyd.conferdeploy.net,dev-prodsyd.conferdeploy.net,cdc-file-storage-production-ap-southeast-

```

2.s3.amazonaws.com,storage.googleapis.com,psc-jobs-prod-output-ap-southeast-
2.s3.ap-northeast-
1.amazonaws.com,updates2.cdc.carbonblack.io,ocsp.godaddy.com,ocsp.digicert.com,crl.
godaddy.com,crl3.digicert.com,crl4.digicert.com,content.carbonblack.io -i
34.117.142.85

--
defense-prodsyd.conferdeploy.net
dev-prodsyd.conferdeploy.net
cdc-file-storage-production-ap-southeast-2.s3.amazonaws.com
storage.googleapis.com
psc-jobs-prod-output-ap-southeast-2.s3.ap-northeast-1.amazonaws.com
updates2.cdc.carbonblack.io
ocsp.godaddy.com
ocsp.digicert.com
crl.godaddy.com
crl3.digicert.com
crl4.digicert.com
content.carbonblack.io
34.117.142.85

--

ludus testing allow -f allowlist.txt
--

ludus --user cbcdemo2 testing start
ludus --user cbcdemo2 testing allow -f allowlist.txt

ludus --user cbcdemo2 testing allow -d defense-prodsyd.conferdeploy.net,dev-
prodsyd.conferdeploy.net,cdc-file-storage-production-ap-southeast-
2.s3.amazonaws.com,storage.googleapis.com,psc-jobs-prod-output-ap-southeast-
2.s3.ap-northeast-
1.amazonaws.com,updates2.cdc.carbonblack.io,ocsp.godaddy.com,ocsp.digicert.com,crl.
godaddy.com,crl3.digicert.com,crl4.digicert.com,content.carbonblack.io -i
34.117.142.85

--
ludus --user cbcdemo2 testing status
ludus --user cbcdemo2 testing stop

--

```

Basic Active Directory Lab

asic Active Directory Network

<https://docs.ludus.cloud/docs/environment-guides/basic-ad-network>

```
rm badn.yaml
```

```
curl -H 'Cache-Control: no-cache' badn.yaml
```

```
"https://raw.githubusercontent.com/RockAfeller2013/ludus/refs/heads/main/Basic%20Active%20Directory%20Network.yaml"
```

```
curl -H 'Cache-Control: no-cache' -o demo1.yaml  
"https://raw.githubusercontent.com/RockAfeller2013/ludus/refs/heads/main/demo1.yaml"  
"
```

```
ludus range config set -f badn.yaml  
ludus range delay  
ludus range status  
ludus range errors
```

```
ludus range logs -f  
ludus power on --name all  
ludus power off --name all
```

```
ludus range destroy  
ludus range rm
```

LUDUS Iptable

The following iptables command can be run on the Ludus host to restrict traffic to the Ludus server to only hosts that are connected via WireGuard.

```
sudo iptables -I INPUT -p tcp --dport 8080 ! -i wg0 -j DROP
```

```
# Persist the rule across reboots  
sudo apt install iptables-persistent  
sudo /sbin/iptables-save > /etc/iptables/rules.v4
```

You may also wish to limit access to the Proxmox web interface (tcp/8006) in the same way.

Export Keys for SSH

```
#Make a bash alias to run the export key every time you login
```

```
# Make user Root  
usermod -aG sudo username
```

```
# Edit the shell configuration file for your user  
nano ~/.bashrc
```

```
# Add the following line at the end of the file  
export LUDUS_API_KEY='SA.5crj8JM%'
```

```
# Save and close the file
```

```
# Reload the shell configuration to apply changes immediately  
source ~/.bashrc
```

routers.iptables_command

upcoming routers.iptables_command feature to permanently allow an IP, regardless of testing status

Range Config

```
# yaml-language-server: $schema=https://docs.ludus.cloud/schemas/range-config.json

ludus:
  - vm_name: "{{ range_id }}-ad-dc-win2019-server-x64" # The name of the VM in
Proxmox. You can use the `{{ range_id }}` template string which resolves to your
range ID (i.e. JS)
    hostname: "{{ range_id }}-DC01-2019" # The hostname for the VM. Note: Windows
host names are limited to 15 characters due to NETBIOS
    template: win2019-server-x64-template # The template that will be the base for
this VM (`ludus templates list` to get a list of them)
    vlan: 10 # The VLAN for this VM. This number will be the third octet
of the VM's IP and must be 2<=vlan<=255
    ip_last_octet: 11 # The last octet for this VM's IP address. Must be unique
in the VLAN.
    force_ip: true # If set to true, the config defined IP address will be
used for the VM when no IP is available via qemu-guest-agent (only use for EDR
appliances or anti-sandbox VMs that do not have qemu-guest-agent installed)
    ram_gb: 8 # The amount of RAM for this VM
    cpus: 4 # The number of cpu cores to allocate to this VM (can
provision more CPUs than the host physically has)
    windows: # This key must be set for windows VMs - all subkeys are
optional
        sysprep: false # Set to true to run sysprep before any other tasks on this
VM. Default: false
        gpos:
            - disable_defender # Create and link a GPO this disables Windows Defender
for every domain joined Windows machine
            - anon_share_access # Create and link a GPO that enables anonymous SMB
share access for every domain joined Windows machine (for use with `ludus range
deploy -t share`)
        domain: # Define this key to put this machine in a domain
            fqdn: ludus.network # The FQDN of the domain
            role: primary-dc # The role of the VM in the domain (primary-dc|alt-
dc|member)
        linux: false # Set this to true for linux VMs - leave undefined or
false for Windows/macOS VMs
        macOS : false # Set this to true for macOS VMs - leave undefined or
false for Windows/Linux VMs
        testing: # This key controls the behavior of the VM in testing
mode. If undefined, both values are set to true
            snapshot: true # Snapshot this VM going into testing, and revert it
coming out of testing. Default: true
            block_internet: true # Cut this VM off from the internet during testing.
Default true
```

```

    roles:                                # This key is an array of user-defined roles that will
be installed on this VM. Roles must exist on the Ludus server and can be installed
with `ludus ansible role add`
    - geerlingguy.docker # Arbitrary role name, as it appears in `ludus ansible
roles list`
    - name: badsectorlabs.ludus_elastic_agent # You can also specify a role with
a an array of dependencies that have to run first
      depends_on:
        - vm_name: "{{ range_id }}-elastic" # In this case, we have to set up
the elastic container (server) before the agent
          role: badsectorlabs.ludus_elastic_container
      role_vars:                          # This key contains `key: value` pairs of variables
that are passed to ALL user-defined roles.
        docker_edition: ce # Arbitrary variables for user-defined roles. Do *not*
use hyphens to prefix these variables, the role_vars key *must* be a dictionary!
        docker_users:      # You can use lists or dicts here
          - localuser
        ansible_groups:     # Define the groups this VM will be a part of when
`ludus range inventory` is run. These groups can be set after VM deployment with
`ludus range deploy -t custom-groups`
          - customgroup1    # An arbitrary group name
          - customgroup2
        dns_rewrites:       # Any values in this array will be added to DNS for the
range and return an A record for this VM's IP
          - example.com     # rewrites responses for this domain name only
          - '*.example.com' # rewrites responses for all example.com subdomains but
*not* example.com
        unmanaged: false   # Set this to true for VMs that cannot report an IP to
ansible via proxmox (no qemu-guest-agent, i.e. EDR appliances). You will have to
manually configure this VM to have the IP defined in the config. Default: false
        - vm_name: "{{ range_id }}-ad-win11-22h2-enterprise-x64-1"
          hostname: "{{ range_id }}-WIN11-22H2-1"
          template: win11-22h2-x64-enterprise-template
          vlan: 10
          ip_last_octet: 21
          ram_gb: 8
          cpus: 4
          windows:
            sysprep: false
            chocolatey_ignore_checksums: false # Set to true to ignore any checksum
errors when installing chocolatey packages (for packages that are 3rd party hosted
and update before the choco package hash updates). Default: false
            chocolatey_packages:
              - vsodium      # An array of chocolatey package names you'd like
installed on this VM. Default: none
              office_version: 2019 # The Microsoft office version you would like
installed on this VM (2013|2016|2019|2021). Default: undefined (don't install
office)
              office_arch: 64bit # The architecture for the Microsoft office
install (64bit|32bit)
              visual_studio_version: 2019 # The version of Microsoft Visual Studio to
install (community edition). Note: 2022 cannot target < .NET 4.5. Default:

```

```

undefined (don't install visual studio)
    autologon_user: myuser      # The username to use for autologon. Default:
localuser unless domain joined, then default.ad_domain_user
    autologon_password: mypass # The password to use for autologon. Default:
password unless domain joined, then default.ad_domain_user_password
    domain:
        fqdn: ludus.network
        role: member
- vm_name: "{{ range_id }}-elastic"
  hostname: "{{ range_id }}-elastic"
  template: debian-12-x64-server-template
  vlan: 20
  ip_last_octet: 2
  ram_gb: 8
  cpus: 4
  linux: true
  roles:
    - badsectorlabs.ludus_elastic_container # This role will run before the
elastic agent role on the windows VM
- vm_name: "{{ range_id }}-kali"
  hostname: "{{ range_id }}-kali"
  template: kali-x64-desktop-template
  vlan: 99
  ip_last_octet: 1
  ram_gb: 8
  cpus: 4
  linux:
    packages: # You can define packages to install on Linux hosts
      - curl
      - python3
  testing:
    snapshot: false
    block_internet: false

# This key contains `key: value` pairs of variables that are passed to ALL user-
defined roles on all VMs in the range
global_role_vars:
  docker_edition: ce

# This key defines the settings for the router VM in your range. It is optional,
and by default Ludus will deploy a router VM with the default settings.
router:
  vm_name: "{{ range_id }}-router-debian11-x64"
  hostname: "{{ range_id }}-router"
  template: debian-11-x64-server-template
  ram_gb: 2
  ram_min_gb: 1
  cpus: 2
  roles: # Enterprise only
    - ludus_guacamole_server # You can add roles to the router VM just like any
other VM
  role_vars: # Enterprise only

```

```

    somevar: value
    outbound_wireguard_config: |- # Enterprise only; Note: this config must have
AllowedIPs set to 0.0.0.0/0, split tunnels are not supported (yet)
    [Interface]
    PrivateKey = XXXX
    ...
    outbound_wireguard_vlans: # Enterprise only; Define the VLANs that the router VM
will route traffic for. This key is required if outbound_wireguard_config is
defined.
    - 10
    inbound_wireguard: # Enterprise only; Must be used with users that have
portforwarding enabled at creation time
    enabled: true # Enable or disable the WireGuard server on the router (default:
false)
    server_cidr: 10.254.254.0/24 # The CIDR of the WireGuard server (default:
10.254.254.0/24)
    port: 51820 # The port the WireGuard server is listening on (UDP) (default:
51820). Note: This must be 51820 to work with port forwarding.
    allowed_vlans: # The VLANs that WireGuard clients are allowed to connect to
(default: all)
    - 10

# This key defines network rules in your range. It is optional, and by default all
traffic is allowed
network:
    inter_vlan_default: REJECT      # The default rule to apply to traffic between
VLANs. Default: ACCEPT
    external_default: ACCEPT        # The default rule to apply to traffic leaving the
range out to the internet. Default: ACCEPT
    wireguard_vlan_default: ACCEPT # The default rule to apply to traffic leaving the
range to clients connected via WireGuard. Default: ACCEPT
    always_blocked_networks:        # Define any networks that ranges should never be
able to reach (i.e. the LAN where the Ludus host is located)
    - 192.168.1.0/24                # entries must be in CIDR format
    rules: # Specify rules to restrict or allow based on inter_vlan_default. Default:
allow all traffic
    - name: Only allow windows to kali on 443 # The rule name will be added as a
comment in iptables
      vlan_src: 10    # Traffic source VLAN. Special value 'public' can be used
which is converted to '! 10.ID.0.0/16' in the iptables rule, 'all' is
'10.ID.0.0/16', and 'wireguard' is '198.51.100.0/24'
      vlan_dst: 99    # Traffic destination VLAN. Special value 'public' can be used
which is converted to '! 10.ID.0.0/16' in the iptables rule, 'all' is
'10.ID.0.0/16', and 'wireguard' is '198.51.100.0/24'
      protocol: tcp    # Protocol (tcp|udp|icmp|all)
      ports: 443       # A single port, a range in the format start:end, or 'all'
      action: ACCEPT # ACCEPT|REJECT|DROP
    - name: Allow kali to the DC
      vlan_src: 99
      ip_last_octet_src: 1 # The single machine from the vlan_src to apply the rule
to
      vlan_dst: 10

```

```

    ip_last_octet_dst: 11 # The single machine from the vlan_dst to apply the
rule to
    protocol: all
    ports: all
    action: ACCEPT

# These values control the values Ludus uses when deploying ranges and Windows
domains.
# The values shown here are the defaults that will be set if this key is not
defined
# If you define the defaults key, you must set every value as it overrides the
server set defaults dict
# Users may wish to change these values to emulate environments (functional level,
domain admin username and password, etc)
defaults:
    snapshot_with_RAM: true # When entering testing mode, capture
the RAM state which allows reverting to a running VM
    stale_hours: 0 # How many hours until a pre-existing
snapshot should be deleted and retaken (if entering and exiting testing mode
quickly)
    ad_domain_functional_level: Win2012R2 # The functional level of each Windows
domain created by Ludus - options are: "Win2003", "Win2008", "Win2008R2",
"Win2012", "Win2012R2", "WinThreshold", or "Win2025"
    ad_forest_functional_level: Win2012R2 # The functional level of each Windows
forest created by Ludus - options are: "Win2003", "Win2008", "Win2008R2",
"Win2012", "Win2012R2", "WinThreshold", or "Win2025"
    ad_domain_admin: domainadmin # The domain admin username for every
Windows domain
    ad_domain_admin_password: password # The domain admin password for every
Windows domain
    ad_domain_user: domainuser # The domain user username for every
Windows domain
    ad_domain_user_password: password # The domain user password for every
Windows domain
    ad_domain_safe_mode_password: password # The domain safe mode password for
every Windows domain
    timezone: America/New_York # The timezone for all VMs, use the TZ
identifier format from https://en.wikipedia.org/wiki/List\_of\_tz\_database\_time\_zones
    enable_dynamic_wallpaper: true # Enable dynamic wallpaper (red/green)
for all Windows VMs in the range

# Use the notify key to send a notification when a range deployment finishes or
fails.
# Notification "urls" look like the following. For more information about service
specific configuration see
# Shoutrrr's documentation: https://containrrr.dev/shoutrrr/services/overview/
#
# Note: usernames and passwords containing special characters will need to be
urlencoded.
# If your username is: "myname@example.com" and your password is "124@34$1"
# your shoutrrr url will look like:
"smtpt://myname%40example%2Ecom:124%4034%241@ms.my.domain.com:587"

```

```

notify:
  urls:
    - "discord://token@webhookid"
    - "telegram://token@telegram?channels=channel-1[,channel-2,...]"
    - "pushover://shoutrrr:apiToken@userKey/?priority=1&devices=device1[,device2,...]"
    - "slack://[botname@]token-a/token-b/token-c"
    - "smtp://username:password@host:port/"
fromAddress=fromAddress&toAddresses=recipient1[,recipient2,...]"
  - "teams://token-a/token-b/token-c"
  - "gotify://gotify-host/token"
  - "pushbullet://api-token[/device/#channel/email]"
  - "ifttt://key/"
events=event1[,event2,...]&value1=value1&value2=value2&value3=value3"
  - "mattermost://[username@]mattermost-host/token[/channel]"
  - "ntfy://username:password@host:port/topic"
  - "hangouts://chat.googleapis.com/v1/spaces/F00/messages?key=bar&token=baz"
  - "zulip://bot-mail:bot-key@zulip-domain/?stream=name-or-id&topic=name"
  - "join://shoutrrr:api-key@join/?devices=device1[,device2,...][&icon=icon]
[&title=title]"
  - "script:///file/path/on/disk"
  - "https://www.example.com/path"

```

Proxmox Ludus.cloud Helper Script

<https://github.com/UntouchedWagons/Ubuntu-CloudInit-Docs/tree/main/samples/debian>

Install Ludus

```
## Build Ludus
```

```
## Requirements
```

https://www.youtube.com/watch?v=RyKjnswA_Mo

x86_64 (aka amd64 aka 64-bit "Intel")

Debian 12

Supports nested virtualization is supported

32 GB of RAM

400 GB of disk space (fast NVMe recommended)

ISO bootable

```
# ISO
```

<https://cloud.debian.org/images/cloud/bookworm/latest/>

<https://cloud.debian.org/images/cloud/bookworm/latest/debian-12-genericcloud-amd64.qcow2>

```
curl -o /var/lib/vz/template/iso/debian-12.11.0-amd64-netinst.iso
```

```
https://cdimage.debian.org/debian-cd/current/amd64/iso-cd/debian-12.11.0-amd64-netinst.iso
```

```
qm create 9004 --name Ludus2 --description "" --boot "order=ide2;scsi0" --cores 16
--cpu host --machine q35 --bios ovmf --ide2 local:iso/debian-12.11.0-amd64-
```

```
netinst.iso,media=cdrom, --memory 32768 --net0 virtio,bridge=vmbro,firewall=1 --
numa 0 --ostype l26 --scsi0 local-lvm:500,iothread=1 --scsihw virtio-scsi-single --
sockets 1 --efidisk0 local-lvm:1,efitype=4m,format=raw --bootdisk scsi0 --agent
```

```
qm create 9004 --name Ludus2 --description "" --boot "order=ide2;scsi0" --cores 16
--cpu host,+vmx --machine q35 --bios ovmf --ide2 local:iso/debian-12.11.0-amd64-
netinst.iso,media=cdrom --memory 32768 --net0 virtio,bridge=vmbro,firewall=1 --numa
0 --ostype l26 --scsi0 local-lvm:vm-9004-disk-0,size=500G,iothread=1 --scsihw
virtio-scsi-single --sockets 1 --efidisk0 local-lvm:1,efitype=4m,format=raw --
bootdisk scsi0 --agent enabled=1
```

```
qm set 9004 --agent enabled=1
```

```
pvesm status
```

```
ps -auwx | grep <VMID>
```

```
#####
```

```
# Run this on the Physical Console, otherwise the Install GUI doesnt work.
```

```
# Debian 12 disables root SSH login by default for security. Here are a few ways
```

```
#####
```

```
## SSH into the VM
```

```
ssh-keygen -R 192.168.1.28
```

```
ssh root@192.168.1.38
```

```
scp * root@19@192.168.1.30/user/root
```

```
usermod -aG sudo ghost
```

```
su - ghost
```

```
su -
```

```
apt update
```

```
sudo apt install -y qemu-guest-agent
```

```
sudo systemctl enable --now qemu-guest-agent
```

```
systemctl status qemu-guest-agent
```

```
grep -E 'vmx|svm' /proc/cpuinfo
```

```
apt install -y libvirt-clients
```

```
virt-host-validate
```

```
# Basic Checks
```

```
ip -4 -o addr show | awk '{print $4}' | cut -d/ -f1 | paste -sd ' '
```

```
cat /etc/os-release
```

```
uname -r
```

```
cat /etc/resolv.conf
```

```
hostname
```

```
su -
```

```
apt update && apt install curl sudo -y
```

```

curl -s https://ludus.cloud/install | bash

systemctl restart ludus
systemctl restart ludus-admin
#####

ludus version --url https://127.0.0.1:8080

#####
# Run this on the Physical Console, otherwise the Install GUI doesnt work.
# Debian 12 disables root SSH login by default for security. Here are a few ways to
fix or work around it:

su -
apt update && apt install curl sudo -y

curl -s https://ludus.cloud/install | bash

systemctl restart ludus
systemctl restart ludus-admin
#####

LUDUS_API_KEY='ROOT.-ePlwMRY-Hh-e@k==KC%crsWAafdvlYzdC6wk4=C' \
  ludus user add --name "serveradmin" --userid SA --admin --url
https://127.0.0.1:8081

SA      | serveradmin      | true  | SA.%q9XLwlljVqVH5wKHh3VfcfvrKz-WCi+NAN1ym9k

```

```
#####
```

Run script

<https://github.com/Orange-Cyberdefense/G0AD/blob/aa7f619ee01f95ff600e105b0f24c37d027af767/ansible/roles/settings/keyboard/tasks/main.yml>

https://github.com/badsectorlabs/ludus_ansible_role_template

Ludus Cheat Sheet

```
# Ludus Basic Network commands
```

```
ludus version --url https://127.0.0.1:8080
```

```
# Enable testing mode
```

```

ludusctl testing enable

# Disable testing mode
ludusctl testing disable

# Create the domain controller
ludusctl create vm "{{ range_id }}"-ad-dc-win2022-server-x64"

# Create the Windows 11 member workstation
ludusctl create vm "{{ range_id }}"-ad-win11-22h2-enterprise-x64-1"

# Create the Kali Linux attacker VM
ludusctl create vm "{{ range_id }}"-kali"

# Destroy all VMs in the environment
ludusctl destroy all

# Show environment status
ludusctl status

# SSH into the Kali VM (replace <range_id> with your actual range)
ssh administrator@"{{ range_id }}"-kali

# Run Windows commands on the domain controller or Windows client (via RDP or other
method)
# Example PowerShell commands to check domain info:
Get-ADDomain
Get-ADUser -Filter *

# To reset the environment
ludusctl reset

# List all VMs managed by Ludus
ludusctl list vms

# Connect to Windows VM via RDP (example)
mstsc /v:"{{ range_id }}"-WIN11-22H2-1

# Snapshot a VM (example)
ludusctl snapshot create "{{ range_id }}"-WIN11-22H2-1" snapshot1

# Restore a snapshot
ludusctl snapshot restore "{{ range_id }}"-WIN11-22H2-1" snapshot1

# Ludus CLI Intro commands

# Authentication & Configuration
ludus apikey <API_KEY>          # Store your Ludus API key in the system keyring
ludus user creds get             # Retrieve Proxmox credentials for the user
ludus version                    # Display the version of the Ludus client

```

```

# Range Management
ludus range config get      # View the current range configuration
ludus range config set     # Set a new range configuration
ludus range deploy         # Deploy the range based on the configuration
ludus range destroy        # Destroy the deployed range

ludus range deploy --limit <vm name>

ludus range deploy -t user-defined-roles --limit localhost,LVZ-WIN11 --only-roles
ludus_french_keyboard_set && ludus range logs -f

# Template Management
ludus templates add -d <dir>    # Add a new template from a directory
ludus templates list           # List all available templates
ludus templates build -n <name> # Build a template by name
ludus templates logs -f        # Follow the build logs of a template

# Testing Mode
ludus testing enable          # Enable testing mode
ludus testing disable         # Disable testing mode

# User Management
ludus users list              # List all users
ludus users create <name>     # Create a new user
ludus users delete <name>     # Delete an existing user

# WireGuard VPN
ludus user wireguard          # Retrieve WireGuard configuration for the user
ludus user wireguard --user <ID> # Retrieve WireGuard configuration for another
user

# Snapshot Management
ludus snapshots list          # List all snapshots for VMs in the range
ludus snapshots create <name> # Create a new snapshot with the given name
ludus snapshots restore <name> # Restore a snapshot by name

ludus --user cbcdemo1 snapshot create mycoolsnapshot

# Ansible Integration
ludus ansible add <role>      # Add an Ansible role to the Ludus server
ludus ansible remove <role>   # Remove an Ansible role from the Ludus server

# Power Management
ludus power on <vmid>         # Power on a VM by ID
ludus power off <vmid>        # Power off a VM by ID
ludus power reboot <vmid>     # Reboot a VM by ID

# Snapshots
qm list
qm snapshot 100 BeforeUpdate --vmstate
qm rollback 100 BeforeUpdate

```

```
qm delsnapshot 100 BeforeUpdate
qm listsnapshot 100
qm list | awk '{print $1" "$2}'

# Remove packer cache to save space
rm -rf /opt/ludus/users/serveradmin/packer/packer_cache#/*

/opt/ludus/<user>/packer/packer_cache
```

Template Rebuild

```
ludus templates list | cut -d ' ' -f 2 > all_templates.txt

xargs -a all_templates.txt -n1 -I{} ludus templates rm "{}"

for i in $(ls); do ludus templates add -d $i --force; done

for DIR in $(ls .); do if [[ "$DIR" == "manual-setup-required" || "$DIR" ==
"README.md" ]]; then continue; else echo "Adding $DIR" && ludus templates add -d
$DIR; fi; done
```

```
gitlab-runner@ludus:/root$ ludus templates status
```

TEMPLATE BEING BUILT	USER
debian-11-x64-server-template	continuous-integration-admin
debian-12-x64-server-template	continuous-integration-admin
kali-x64-desktop-template	continuous-integration-admin
win11-22h2-x64-enterprise-template	continuous-integration-admin

Ludus Range deployment

Useful Commands

```
shutdown now

ludus range status
ludus power on --name all
ludus power off --name all
ludus range errors
ludus range rm
ludus range deploy
ludus range config get
ludus range logs -f

ludus range config get > ludus-range-config.yml
ludus range config set -f ludus-range-config.yml
```

When testing mode is enabled, both Windows hosts will be snapshotted and blocked from reaching the internet, while the Kali VM will not be snapshotted or blocked from reaching the internet.

Testing

```
ludus testing start
ludus testing status
```

Allow Domains

```
ludus testing allow -d example.com
```

Access Wireguard

```
https://www.wireguard.com/install/
```

wireguard Troubleshooting

```
https://chatgpt.com/c/67c51e07-f06c-800e-81e3-4f316325efff
```

```
wg show
wg showconf
dmesg
journalctl -xe
wg-quick down wg0 && wg-quick up wg0
```

```
/etc/wireguard/wg0.conf
```

```
wg genkey
wg pubkey < privatekeyfile > publickeyfile
```

```
ludus user wireguard | tee ludus.conf
user wireguard --user <UserID>
Save as ludus.confg
```

```
https://192.168.193.140:8006/
```

```
https://10.2.99.1:8444/
```

Default Machine Credentials

```
kali:kali
Windows - Administrator:password
LUDUS\domainuser:password
LUDUS\domainadmin:password (Domain Admin)
debian:debian
localuser:password
```

Snapshotting

```
ludus --user cbcdemo1 power off --name all
ludus --user cbcdemo1 snapshots create pre-license -d "Install windows license"
ludus --user cbcdemo1 snapshots revert pre-license
ludus --user cbcdemo1 snapshots list
```

Use your own Windows ISO and License Keys

WIP

Troubleshooting

Networking Not working

Networking Not working

```
systemctl restart networking
```

```
iptables -I FORWARD 1 -i vmbr1000 -o eno1 -j ACCEPT
iptables -I FORWARD 1 -i eno1 -o vmbr1000 -m state --state RELATED,ESTABLISHED -j
ACCEPT
```

```
sysctl net.ipv4.ip_forward
```

dmesg will show OOM kills to the qemu processes

```
dmesg | grep -i "out of memory\|oom"
```

```
dmesg -T | egrep -i 'killed process'
```

Activate Windows MSDN License keys (Not possible.)

Install Windows License

It is not possible to activate these templates to production windows licenses or use your MSDN license keys.

'MSDN key is Retail: requires non-eval version of Enterprise. ' tried to convert it, but still no go..

```
DISM /online /Set-Edition:Enterprise /ProductKey:XXXXX-XXXXX-XXXXX-XXXXX-XXXXX  
/AcceptEula
```

UI Issues

```
apt install --reinstall proxmox-widget-toolkit
```

Hostnames are truncated

```
[ERROR] Bad Request  
[ERROR] Configuration error: duplicate Windows hostname name found: {{ range_id }}-  
WIN10-22H2-2  
Windows hostnames are truncated to 15 characters so the first 15 characters must be  
unique  
  
## Shorten the hostname!!
```

Chocolatey errors

Some chocolatey packages pull from external sources and do not update their checksums frequently.
Consider setting `chocolatey_ignore_checksums: true` in your range configuration for this VM to ignore checksums and bypass this error.

Read - <https://docs.ludus.cloud/docs/configuration/>

```
chocolatey_ignore_checksums: true in
```

Ludus (apt install qemu-server=8.3.12 -y)

```
## Error building Templates
```

```
proxmox-iso.win2019-server-x64-no-security-updates: Error converting VM to  
template, could not stop: VM quit/powerdown failed - got timeout
```

```
## Fix  
su -  
apt install qemu-server=8.3.12 -y  
ludus template build
```

Troubleshooting LUDUS Install

Troubleshooting LUDUS install

Sometimes the vmbr1000 NIC will go down, you can just use the following or reboot and kick off the install again.

```
systemctl restart networking
```

```
iptables -I FORWARD 1 -i vmbr1000 -o eno1 -j ACCEPT
```

```
iptables -I FORWARD 1 -i eno1 -o vmbr1000 -m state --state RELATED,ESTABLISHED -j ACCEPT
```

```
sysctl net.ipv4.ip_forward
```

dmesg will show OOM kills to the qemu processes

```
dmesg | grep -i "out of memory\|oom"
```

```
dmesg -T | egrep -i 'killed process'
```

Enable Root SSH login to Debian 12/Ludus

```
# Switch to root or use sudo
```

```
sudo -i
```

```
# Edit the SSH server config
```

```
nano /etc/ssh/sshd_config
```

```
# Find the line:
```

```
# PermitRootLogin prohibit-password
```

```
# Change it to:
```

```
PermitRootLogin yes
```

```
# Save and exit (in nano: Ctrl+O, Enter, Ctrl+X)
```

```
# Set a root password if not already set
```

```
passwd
```

```
# Restart SSH service to apply changes
```

```
systemctl restart ssh
```

Add Ludus Templates

```
+-----+-----+
```

```
# Adding Templates
```

```
ludus templates list
```

```
+-----+-----+
```

```
|          TEMPLATE          | BUILT |
```

```
+-----+-----+
```

```
| debian-11-x64-server-template | FALSE |
```

```
| debian-12-x64-server-template | FALSE |
```

```
| kali-x64-desktop-template    | FALSE |
```

```
| win11-22h2-x64-enterprise-template | FALSE |
| win2022-server-x64-template         | FALSE |
+-----+-----+
```

```
ludus templates build -n debian-11-x64-server-template
```

```
+-----+-----+
```

```
git clone https://gitlab.com/badsectorlabs/ludus
cd ludus/templates
```

```
ludus ansible role add badsectorlabs.ludus_commandvm
ludus ansible role add badsectorlabs.ludus_flarevm
ludus ansible role add badsectorlabs.ludus_remnux
```

```
for x in $(ls); do ludus templates add -d $x; done
ludus templates add -d <template name> -f
```

```
for DIR in $(ls .); do if [[ "$DIR" == "manual-setup-required" || "$DIR" ==
"README.md" ]]; then continue; else echo "Adding $DIR" && ludus templates add -d
$DIR; fi; done
```

```
ludus template list
ludus templates add -d ubuntu-20.04-x64-fr-server -f
ludus templates build -n ubuntu-20.04-x64-fr-server-template
ludus templates build -n debian-11-x64-server-template
ludus templates logs -fv
ludus template status
```

```
## Remove templates
for DIR in $(ls .); do [[ "$DIR" == "manual-setup-required" || "$DIR" ==
"README.md" ]] && continue || (echo "Adding ${DIR}-template" && ludus templates rm
-n "${DIR}-template"); done
```

LudusHound

```
# Ludus Hound
```

Ludus Hound - <https://specterops.io/blog/2025/07/14/ludushound-raising-bloodhound-attack-paths-to-life/>

```
# Ludus Bloodhound CE Ansible Role
```

```
https://github.com/badsectorlabs/ludus\_bloodhound\_ce
```

```
add to the LudusHound Configuration
```

Security

!! WARNING !! Disable Range VM access to local Network

I didn't realise that the Range VM can access the local LAN, even though Ludus was a VM inside Proxmox, I only stumbled upon this by accident, reading thedocs and Discord. This is a massive risk, !! They are going to make this as a default in the future.

Add the following into range config yaml file.

Always Blocked Networks

If you wish to always prevent range networks from reaching a specific network, such as the LAN the Ludus host is deployed on, you can define a list of CIDRs that the range router will prevent access to in the `always_blocked_networks` key in your range config.

```
network:
  always_blocked_networks:  # Define any networks that ranges should never be able
                           # to reach (i.e. the LAN where the Ludus host is located)
                           - 192.168.1.0/24      # entries must be in CIDR format
```

<https://docs.ludus.cloud/docs/security/>

The following iptables command can be run on the Ludus host to restrict traffic to the Ludus server to only hosts that are connected via WireGuard.

```
sudo iptables -I INPUT -p tcp --dport 8080 ! -i wg0 -j DROP
```

```
# Persist the rule across reboots
sudo apt install iptables-persistent
sudo /sbin/iptables-save > /etc/iptables/rules.v4
```

You may also wish to limit access to the Proxmox web interface (tcp/8006) in the same way.

Malicious Users

Giving users the ability to add arbitrary Ansible roles is effectively allowing for remote code execution, as a role could simply be a reverse shell executed on host localhost (the Ludus host). The flexibility offered by arbitrary Ansible roles is worth the security trade off for nearly all use cases.

To prevent non-admin users from adding Ansible roles or collections, set the following option in the Ludus config file.

```
/opt/ludus/config.yml
prevent_user_ansible_add: true
```

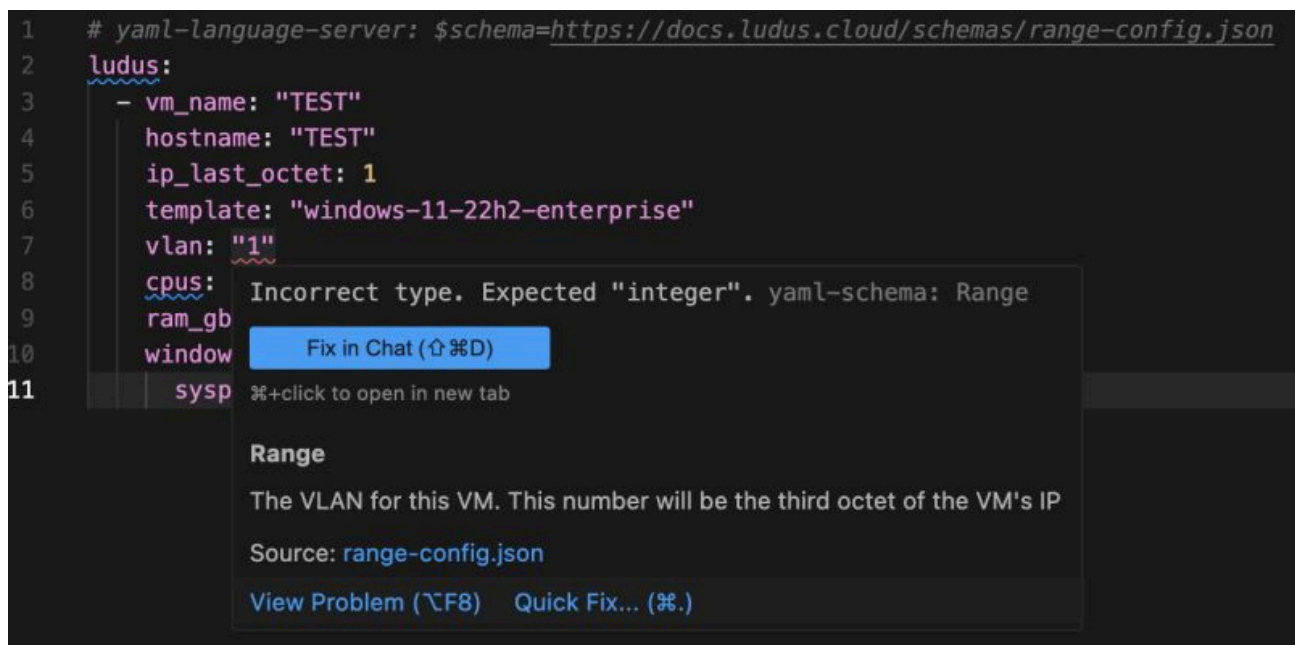
The Ludus server process (port 8080), is heavily sandboxed (i.e. all files outside of /opt/ludus are read only), however there are likely still methods of privilege escalation.

User's router VMs are configured to allow traffic from the Ludus host, and the user's WireGuard IP. A user could write an Ansible module to execute as localhost and interact with other user's router VMs.

```
-----
# block any range VM from accessing the host by default
https://gitlab.com/badsectorlabs/ludus/-/commit/40bb4cccbe5817271b6d08e9a1825e7f6750481d
```

The Ludus range config can get complex – lots of features == lots of options, but VSCode (and Cursor/Windsurf) can help if you add:

#yaml-language-server: \$schema=https://docs.ludus.cloud/schemas/range-config.json



LUDUS Access

<https://10.2.99.1:8444/>

<https://192.168.193.140:8006/#v1:0:18:4:.....>

Configure Tailscale

https://github.com/NocteDefensor/ludus_tailscale?tab=readme-ov-file

[Watch on YouTube](#)

[Watch on YouTube](#)

[Watch on YouTube](#)

Sharing Ludus

To Do

- Remote access via
- Install Guacomlate
- Install Tailscale or Cloudflare tunnels or Cloudflare Apps
- <https://kasmweb.com/app-isolation>
- Install <https://parsec.app/>
- Pangolin Logo – <https://github.com/fosrl/pangolin>
- Moonlight and Sunshine – https://www.reddit.com/r/MoonlightStreaming/comments/1dimapk/remote_moonlight_sunshine_tutorial/
- Setup the hack lab
- Setup Tailscale
- Deploy the CBC agent
- Convert a Working range to a Template

Cost

- Host a Ludus server in Sydney that would support 5 users running the basic active directory network for \$450 USD per month. This would be the community edition of Ludus, with email based support.
- Access to the server is provided via WireGuard.

Gotchas

- Make sure you use the correct ISO, link in the document. I downloaded a 32 bit ISO.
- Make sure CPU supports virtualisation. If you are trying to run this inside VMware Workstation, your CPU needs to support Nested Virtualization.
- Setup Storage for /opt with more than 500 GB, as per <https://help.ubuntu.com/community/Partitioning/Home/Moving>
- Use the Ludis Client

- Use Wireguard
- Use tailscale, cloudflare and Guacamole server secure access – <https://www.youtube.com/watch?v=ejWwJQvqiW>
- When you run the first LUDUS_API_KEY Command, make sure there is a single space in front of the command and just edit /copy and paste it exactly. If there is not space, command fails. I lost a few brains cells because of this.
- Add the API command to startup
- KALI issues *“Before deploying, a few prereqs are worth considering for your templates(Fix needed below at the time of writing Dec 2024). In my case, I had to comment out the provisioner section of /opt/ludus/packer/kali/kali.pkr.hcl due to the bug referenced in the Ludus documentation*
- Setup the Cache setting
- Cron delete cache

References

- <https://blog.zsec.uk/homelab-clustering-pt1/>
- https://www.youtube.com/watch?v=srrKsr_29OI
- <https://www.youtube.com/watch?v=zmytgzu-iME>
- <https://gitlab.com/badsectorlabs/ludus>
- <https://docs.ludus.cloud/docs/quick-start/build-templates>
- GOAD – <https://github.com/Orange-Cyberdefense/GOAD>
- AD Pwnage: Ninja Hackers Academy Part 1 – <https://crypt0ace.github.io/posts/NHA-Part-1/>
- The Ninja Hacker Academy – A full guide to your graduationThe Ninja Hacker Academy – A full guide to your graduation – <https://www.shadowbumble.nl/posts/NHA/>
- GOAD v2 – <https://mayfly277.github.io/posts/GOADv2/>
- LUDS HowT – <https://www.youtube.com/playlist?list=PL1CNTghxc2A20IPcCpuk8pcxBwfsXN3fE>
- AD Pwnage: Ninja Hackers Academy Part 1 – <https://crypt0ace.github.io/posts/NHA-Part-1/>
- The Ninja Hacker Academy – A full guide to your graduation – <https://www.shadowbumble.nl/posts/NHA/>
- DEF CON 32 – Winning the Game of Active Directory – Brandon Colley – <https://www.youtube.com/watch?v=M-2d3sM3I2o&t=79s>
- Slides <https://media.defcon.org/DEF%20CON%2032/DEF%20CON%2032%20villages/DEF%20CON%2032%20-%20Packet%20Hacking%20Village%20-%20Brandon%20Colley%20-%20Winning%20the%20Game%20of%20Active%20Directory.pdf>
- <https://chat.deepseek.com/a/chat/s/07f94323-7359-4c48-94e6-1ee2ae1b385d>

- Ansible roles – https://galaxy.ansible.com/ui/standalone/roles/?page=1&page_size=10&sort=-created&keywords=ludus
- Gucamole – <https://theko2fi.medium.com/deploy-apache-guacamole-with-ansible-and-docker-e46c19273815>
- Ludus Discord – <https://discord.com/channels/1216801602518782042/1216801603030618369>
- <https://course.constructingdefense.com/view/courses/constructing-defense/2361321-lab-construction-aws-version/7515881-terraform-setup>
- <https://www.thenasguy.com/2024/08/12/proxmox-storage-best-practices-optimize-your-setup/>
- Ludus Hound – <https://specterops.io/blog/2025/07/14/ludushound-raising-bloodhound-attack-paths-to-life/>
- Ranges – <https://github.com/jessefmoore/Ludus-Ranges/tree/main/ranges>
- Ludus Bloodhound CE – https://github.com/badsectorlabs/ludus_bloodhound_ce